

Intel Image Classification

From Scratch CNN to Transfer Learning — A Complete Project Documentation

Author: Dev

M.Sc. Statistics, University of Delhi (B.Sc. Statistics, BHU)

Project Duration: 4 Days – 07/06 – 10/06

Dataset: Intel Image Classification (Kaggle)

Repository: github.com/Dev-2004-DA/Intel-Image-Classification

1. Project Motivation & Dataset Choice

The goal of this project was to build an end-to-end image classification pipeline progressing from a custom-built Convolutional Neural Network (CNN) to Transfer Learning with pretrained architectures, while documenting the reasoning behind every architectural and training decision — not just the final accuracy numbers.

Why this dataset?

The Intel Image Classification dataset has 6 scene categories — buildings, forest, glacier, mountain, sea, and street — none of which are standard ImageNet classes. This makes it a genuinely fair benchmark: pretrained ImageNet models do not get a “free pass” on these exact classes, so any improvement from Transfer Learning reflects genuinely transferable low- and mid-level visual features (edges, textures, shapes) rather than memorized class labels.

Dataset Structure

Split	Image Count	Purpose
seg_train/	~14,000	Labelled training images across 6 class folders
seg_test/	~3,000	Labelled test images for evaluation
seg_pred/	~7,301	Unlabelled images, used for agreement-based analysis

- Image size: $150 \times 150 \times 3$ (RGB)
- Classes (alphabetical, as inferred by Keras): buildings (0), forest (1), glacier (2), mountain (3), sea (4), street (5)
- Roughly class-balanced across the training set

2. Environment Setup

Platform: Google Colab (T4 GPU)

2.1 Loading the Dataset from Kaggle

- Generate kaggle.json API token from Kaggle account settings
- Upload via files.upload() in Colab
- Move to ~/.kaggle/, set permissions with chmod 600
- Run: !kaggle datasets download -d puneet6060/intel-image-classification
- Unzip — note the double-nested folder structure (seg_train/seg_train/...), a common gotcha

2.2 Loading Images as a Dataset

Used `tf.keras.utils.image_dataset_from_directory()`, which automatically infers labels from subfolder names and returns batched (image, label) tensors of shape (batch_size, 150, 150, 3).

Note: `image_dataset_from_directory` defaults to `shuffle=True`. This caused a major bug later in the model comparison phase — see Section 6.

3. Model 1 — CNN from Scratch

3.1 Architecture Design Principles

Established the standard convolutional block pattern through iterative discussion and reasoning:

Conv2D (no activation) → BatchNormalization → Activation(ReLU) → MaxPooling → SpatialDropout

Key reasoning behind this ordering:

- BatchNorm is placed before activation so it normalizes the raw convolution outputs before ReLU clips negative values — this preserves more of the signal distribution for BatchNorm to act on.
- Filter count increases with depth (16 → 64 → 128). Early layers detect simple features (edges, gradients); deeper layers detect complex features (textures, shapes, object parts) — critical for scene classification, where texture differences (e.g., glacier ice vs. mountain rock) matter most.
- SpatialDropout2D (not regular Dropout) is used after pooling in conv blocks — it drops entire feature maps rather than individual pixels, which is more effective for spatial/convolutional data.
- A Rescaling (1/255 normalization) layer is placed first, before any computation, to ensure the network operates on normalized pixel values from the start.

3.2 Hyperparameter Tuning

Used Keras Tuner across two independent tuning runs:

- Run 1: tuned filters, kernel sizes, pool size, and dropout rate
- Run 2: same search space, with L2 regularization additionally applied on dense layers

Result: Both runs converged to the same ceiling of 77.8% validation accuracy, regardless of whether L2 regularization was applied.

***Note:** When two structurally different regularization setups produce the same accuracy ceiling, the architecture itself — not the hyperparameters — is the bottleneck. This was the key diagnostic insight that justified moving to Transfer Learning.*

3.3 Final Architecture

```
Input(150,150,3)
→ RandomFlip, RandomRotation(0.3), RandomZoom(0.2), RandomTranslation(0.2)
→ Conv2D(16, 4x4) → Conv2D(64, 4x4) → BatchNorm → ReLU → MaxPool(2x2) → SpatialDropout(0.2)
→ Conv2D(128, 3x3) → BatchNorm → ReLU → MaxPool(3x3)
→ Conv2D(128, 3x3) → BatchNorm → ReLU → MaxPool(3x3) → SpatialDropout(0.2)
→ GlobalMaxPool2D
→ Dense(128, ReLU) → BatchNorm → Dense(64, ReLU) → BatchNorm → Dense(6, Softmax)
```

Total parameters: ~265,878

3.4 Training & Callbacks

- Optimizer: RMSprop, lr = 0.001 (best from tuner — outperformed Adam)
- Callbacks: EarlyStopping (val_loss), ReduceLROnPlateau, ModelCheckpoint (val_accuracy), TensorBoard

Debugging Notes

- Initial EarlyStopping configuration used two simultaneous callbacks — one on val_loss, one on val_accuracy with min_delta=0.1. This was overly aggressive and caused premature stopping. Simplified to a single val_loss-monitored EarlyStopping with restore_best_weights=True and min_delta reduced from 0.02 to 0.001.
- ModelCheckpoint typo bug: an early version had monitor='val_accurcay' (typo), which silently failed to save the best model. Fixed to 'val_accuracy'.

3.5 Final Result

Metric	Value
Train Accuracy	80.27%
Test Accuracy	77.80%
Generalization Gap	2.5% (healthy)

This established the scratch CNN ceiling and justified the move to Transfer Learning.

4. Model 2 — MobileNetV2 (Transfer Learning)

4.1 Architecture

```
Input(150,150,3)
→ RandomFlip, RandomRotation(0.3), RandomZoom(0.2), RandomTranslation(0.2)
→ MobileNetV2(include_top=False, input_shape=(150,150,3)) – last 20 layers unfrozen
→ MaxPooling2D(3x3) → Flatten
→ Dense(128, ReLU) → Dense(64, ReLU) → Dense(6, Softmax)
```

Total parameters: 2,430,598 (Trainable: 1,225,094 / Non-trainable: 1,205,504)

Early Debugging

- MobileNetV2(input_shape=(150,150), include_top=True) raised a ValueError, because include_top=True forces (224,224,3) input — the shape expected by the original ImageNet classification head.
- Fixed by using include_top=False with input_shape=(150,150,3). This produces a UserWarning about non-standard input size but loads pretrained weights correctly via interpolation.
- With include_top=False, the base model output was a 3D feature map of shape (None, 5, 5, 1280). Directly adding Dense(6) on top threw a rank-mismatch error (“output must have rank target.ndim - 1”). Fixed by adding a Flatten() layer before the dense head.

4.2 Progressive Fine-Tuning — 3 Runs

Run	Unfrozen Layers	Train Acc	Val Acc	Verdict
1	Mostly frozen	—	~77%	Underfitting
2	More unfrozen	~96%	~82%	Overfitting (14% gap)
3	Last 20 layers	88.37%	90.57%	Best — strong generalization

4.3 Run 3 — Detailed Training Log Analysis

- Optimizer: RMSprop (default lr = 0.001)
- Regularization: Data Augmentation only (no dropout/L2 — augmentation served as the sole regularizer)
- 30 epochs max, EarlyStopping (patience=6, monitor=val_loss), ReduceLROnPlateau (patience=5)

Epoch-by-Epoch Highlights

- Epoch 1: val_accuracy = 83.6% — immediately strong, showing pretrained features are powerful even for non-ImageNet classes
- Epochs 2–3: val accuracy dipped to 76–77% with high val_loss (1.17–1.38) — initial instability at lr = 0.001
- Epoch 6: ReduceLROnPlateau triggered — lr dropped from 0.001 to 0.0001
- Epochs 7–8: sharp jump — val accuracy rose from 89.6% to 90.4%
- **Epoch 12: best val accuracy of 90.57% (train: 88.37%, val_loss: 0.3211) — saved by ModelCheckpoint**
- Epoch 17: EarlyStopping triggered — model converged

Final Result

Metric	Value
Train Accuracy	88.37%
Validation / Test Accuracy	90.57%
Generalization Gap	-2.2% (val > train)

Note: Validation accuracy consistently exceeded training accuracy from epoch 7 onward. This is not an error — data augmentation made the training distribution harder than the clean (unaugmented) validation/test set, which is a sign of excellent generalization.

5. Model 3 — Xception (Transfer Learning)

5.1 Model Selection Reasoning

After Model 2, the goal was a third architecture that was:

- Architecturally distinct from MobileNetV2 (Xception is much deeper, with residual connections, despite also using depthwise separable convolutions)
- Reasonably sized for hosting alongside the project (Xception base ≈ 88MB; with the custom head, the saved model came to ~112MB — just over GitHub's 100MB limit, handled via a Google Drive link)

EfficientNet Attempts (Abandoned)

EfficientNetB0, B3, and V2B3 were all attempted first. These architectures are optimized for 224×224 / 300×300 input resolutions. At 150×150, they significantly underperformed — training accuracy

remained stuck at 45–50% even after 15 epochs, across multiple unfreezing configurations (last 15, 30, and 50 of 234 layers).

Note: This was diagnosed as an input resolution mismatch problem, not a tuning problem — EfficientNet's deeper layers are tuned to operate on much larger feature maps than 150x150 input produces. Switched to Xception, which handles 150x150 input far better.

5.2 Architecture

```
Input(150,150,3)
→ RandomFlip, RandomRotation(0.3), RandomZoom(0.2), RandomTranslation(0.2)
→ Xception(include_top=False) – last 20 layers unfrozen
→ GlobalMaxPooling2D
→ Dense(256, ReLU) → BatchNorm → Dense(128, ReLU) → BatchNorm → Dense(64, ReLU) → BatchNorm →
Dense(6, Softmax)
```

5.3 Training

- Optimizer: RMSprop (default)
- Regularization: Data Augmentation
- Callbacks: EarlyStopping (patience=6), ReduceLROnPlateau, ModelCheckpoint, TensorBoard

Result — Converged Remarkably Fast (9 Epochs)

- Epoch 1: val_accuracy already at 88.8%; val_loss (0.327) was lower than train_loss (0.587) from the very start
- **Epoch 7: best val_accuracy = 91.73% (train: 90.1%, val_loss: 0.2396)**
- Epoch 9: EarlyStopping triggered

Final Result (via model.evaluate())

Metric	Value
Train Accuracy	93.55%
Test Accuracy	91.73%
Generalization Gap	1.82% (clean)

6. Model Comparison Notebook — Methodology & a Critical Bug

6.1 The Shuffle Bug (Important Lesson)

First approach (incorrect):

```
y_test = []
for images, labels in test_data:
    y_test.extend(labels.numpy())
y_test = np.array(y_test)

y_prob3 = xception_model.predict(test_data, verbose=0)
y_pred_xception_model = np.argmax(y_prob3, axis=1)
```

This produced a classification report showing 17% accuracy across all three models — essentially random — despite the models independently evaluating at 78–92%.

Root Cause

test_data was created with the default shuffle=True. Each time the dataset is iterated (once for collecting y_test, once for predict()), TensorFlow re-shuffles the batch order. The labels collected in the first loop and the predictions from the second pass were therefore misaligned — comparing label[i] from one shuffle order against prediction[i] from a completely different shuffle order.

Fix — Collect Labels and Predictions in the Same Loop, Same Iteration

```
y_test = []
y_pred_scratch_model = []
y_pred_mob_net_model = []
y_pred_xception_model = []

for images, labels in test_data:
    y_test.extend(labels.numpy())

    y_prob1 = scratch_model.predict(images, verbose=0)
    y_pred_scratch_model.extend(np.argmax(y_prob1, axis=1))

    y_prob2 = mob_net_model.predict(images, verbose=0)
    y_pred_mob_net_model.extend(np.argmax(y_prob2, axis=1))

    y_prob3 = xception_model.predict(images, verbose=0)
    y_pred_xception_model.extend(np.argmax(y_prob3, axis=1))

y_test = np.array(y_test)
y_pred_scratch_model = np.array(y_pred_scratch_model)
y_pred_mob_net_model = np.array(y_pred_mob_net_model)
y_pred_xception_model = np.array(y_pred_xception_model)
```

This guarantees every batch's labels and all three models' predictions are collected together, in the same order, in a single pass — no possibility of misalignment. After the fix, results matched the independently-computed model.evaluate() accuracies exactly (e.g., Xception: 92% vs. expected 91.73%).

Note: Whenever shuffle=True is set on a tf.data.Dataset, never assume two separate iterations over it will produce the same order. Always collect everything needed from a single pass.

6.2 Function / Concept Reference (Personal Notes)

- for images, labels in test_data: — iterates over the dataset batch by batch, not image by image. Each images/labels pair is one batch.
- .numpy() — converts a TensorFlow Tensor to a regular numpy array (needed for sklearn / matplotlib compatibility)
- .extend() — appends all elements of an iterable individually to a list (vs. .append(), which would nest it)
- np.argmax(preds, axis=1) — for each row (image) in a batch of class probabilities, returns the index of the highest probability — i.e., the predicted class

General pattern for any classification project:

1. Loop through test data in batches
2. Get model predictions (probabilities) per batch

3. Convert probabilities to class labels via `argmax`
4. Collect true and predicted labels into two aligned lists
5. Feed into `confusion_matrix()` / `classification_report()`

7. Confusion Matrices & Classification Reports

7.1 Test Set Results (3,000 images)

Model	Accuracy	Macro F1
CNN from Scratch	78%	0.78
MobileNetV2	91%	0.91
Xception	92%	0.92

7.2 Per-Class F1 Score Progression

Class	Scratch	MobileNetV2	Xception
Buildings	0.70	0.91	0.91
Forest	0.94	0.99	0.99
Glacier	0.76	0.85	0.87
Mountain	0.74	0.85	0.87
Sea	0.78	0.94	0.95
Street	0.76	0.92	0.93

7.3 Confusion Cross-Tabulation — Glacier ↔ Mountain

Model	Glacier → Mountain	Mountain → Glacier
Scratch	68	36
MobileNetV2	49	80
Xception	71	46

Note: This is the single most consistent error pattern across all three architectures — glacier and mountain images frequently share rocky/icy textures and similar color palettes (greys, whites, blues), making them genuinely difficult even for a quick human glance.

7.4 Other Patterns

- Forest is near-perfect across all 3 models (94–99% F1) — the most visually distinct class (dense green texture, no other class resembles it).
- Street → Buildings confusion: Scratch (117), MobileNetV2 (55), Xception (24) — urban scenes naturally overlap; pretrained models handle this much better than the scratch model.

- Buildings shows the largest jump: Scratch (70% F1) → MobileNetV2 / Xception (91% F1) — pretrained ImageNet features (heavily exposed to man-made structures) transfer very effectively here.

8. Evaluation on Unlabelled Prediction Set (7,301 images)

Since seg_pred has no ground truth, inter-model agreement was used as a proxy for reliability. All three models predicted on every image (loaded in batches of 32), and predictions were compared.

8.1 Agreement Distribution

Agreement Level	Count	Percentage
All 3 models agree	5,477	75.0%
2 of 3 agree	1,681	23.0%
All 3 disagree	143	2.0%

75% unanimous agreement across three architecturally very different models (266K to 22M+ parameters) is strong evidence that all three are learning genuine, generalizable visual features rather than memorizing training-set-specific noise.

8.2 Class-wise Prediction Frequency

A grouped bar chart compared how often each model predicted each class across all 7,301 images:

- Class 0 (Buildings): Scratch over-predicts heavily (~1565) vs. MobileNetV2/Xception (~1170) — consistent with the scratch model's street → buildings confusion bias
- Class 2 (Glacier): MobileNetV2 predicts more (~1380) than Scratch (~1000) and Xception (~1180)
- Class 3 (Mountain): Xception predicts more (~1380) than the other two (~1080–1200)
- Classes 1, 4, 5 (Forest, Sea, Street): all three models agree closely — these are the “easy” classes

8.3 Disagreement Analysis — The 143 Total-Disagreement Images

12 of the 143 images (where all three models gave different predictions) were manually inspected, visually labelled, and each model's prediction plus confidence (max softmax probability) was recorded:

#	True Label (visual)	Scratch (conf)	MobileNet (conf)	Xception (conf)
1	Building	mountain (0.890)	sea (1.000)	glacier (0.999)
2	Glacier	sea (0.657)	glacier (0.971)	mountain (0.504)
3	Mountain	sea (0.429)	glacier (0.999)	mountain (0.972)
4	Glacier	buildings (0.353)	glacier (1.000)	mountain (0.557)
5	Mountain	mountain (0.980)	glacier (0.996)	sea (0.947)
6	Sea	sea (0.927)	glacier (0.841)	mountain (0.949)

#	True Label (visual)	Scratch (conf)	MobileNet (conf)	Xception (conf)
7	Sea	street (0.964)	glacier (1.000)	mountain (0.997)
8	Street	street (0.933)	glacier (0.679)	sea (0.971)
9	Sea	buildings (0.749)	mountain (0.961)	sea (0.959)
10	Glacier	buildings (0.733)	glacier (0.661)	mountain (0.978)
11	Glacier	buildings (0.400)	glacier (0.982)	mountain (0.924)
12	Glacier	buildings (0.611)	glacier (0.953)	sea (0.782)

Key Findings

- Confidently wrong predictions exist even in strong models. MobileNet predicted “sea” with 100% confidence on a true Building image, and “glacier” with 99.99–100% confidence on images that were actually Sea/Mountain. High softmax confidence does not guarantee correctness — especially on inputs that fall outside the model's typical training distribution patterns.
- MobileNetV2 shows the most overconfidence on errors — frequently 95–100% confidence even when wrong (rows 1, 3, 4, 6, 7, 11). Xception's confidence on errors is comparatively more moderate (50.4–99.7%, with several in the 50–80% range) — making its mistakes more “calibrated” (lower confidence correlates somewhat with higher likelihood of error).
- The images themselves are genuinely ambiguous. Image 1 is an ornate European arch/monument that doesn't cleanly fit any of the 6 classes; several images contain glacier+mountain+sea simultaneously in the same frame; one shows a palm tree + sea + boat; another is a sunset over water with unusual lighting/color grading. These are edge cases that would likely produce disagreement among human labellers too.

Conclusion: The remaining errors across all three models are predominantly driven by inherent class ambiguity in real-world scene images (multiple valid scene elements in one frame) rather than by model architecture deficiencies. This is a meaningful distinction for understanding deployment-time failure modes — these are not “fixable” errors through more training; they reflect genuine label ambiguity in the underlying task definition.

9. Final Results Summary

Model	Train Acc	Test Acc	Gap	Parameters	Size
CNN from Scratch	80.27%	77.80%	2.5%	~266K	Small
MobileNetV2	88.37%	90.57%	-2.2% (val>train)	2.43M	~14MB
Xception	93.55%	91.73%	1.8%	Larger	~112MB

Note: MobileNetV2 (~14MB) performed nearly on par with Xception (~112MB) — an ~8x size difference for ~1.2% accuracy difference. For deployment-constrained scenarios (e.g., Streamlit Cloud's ~1GB RAM limit), MobileNetV2 is the clear practical choice despite Xception's marginally higher accuracy.

10. File Management & GitHub Considerations

- Model file size limits: Xception's saved .keras file (~112MB) exceeds GitHub's 100MB hard limit. Resolved by hosting on Google Drive (set to "Anyone with the link") and linking from the README, rather than using Git LFS (impractical for manual/web-based GitHub uploads without command-line Git).
- TensorBoard logs: Not pushed to GitHub — too large, not directly viewable on GitHub, and the same information is captured as static accuracy/loss plots saved as PNGs.
- Dataset: Not pushed (400MB+) — README links directly to the Kaggle dataset page instead.
- API key safety: An initial notebook upload exposed Kaggle kaggle.json credentials in a cell output. This was caught before pushing — outputs were cleared via "Clear all outputs" in Colab before any GitHub upload.

Note: Always clear notebook outputs before pushing API-key-containing cells to a public repository.

11. Repository Structure (Final)

```
Intel-Image-Classification/
├── notebooks/
│   ├── 01_CNN_from_scratch.ipynb
│   ├── 02_Pretrained_MobileNetV2.ipynb
│   ├── 03_Xception_Transfer_Learning.ipynb
│   └── 04_Model_Comparison.ipynb
├── models/
│   ├── BEST_MODEL.keras                # CNN from scratch
│   ├── BEST_MODEL_MOBILENET.keras     # MobileNetV2 best
│   └── README.md                      # Xception too large – Drive link inside
├── images/
│   ├── cnn_accuracy.png / cnn_loss.png
│   ├── mobilenet_accuracy.png / mobilenet_loss.png
│   ├── xception_accuracy.png / xception_loss.png
│   ├── confusion_matrices/
│   │   ├── cm_scratch_cnn.png
│   │   ├── cm_mobilenet.png
│   │   └── cm_xception.png
│   └── comparison/
│       ├── disagreement_images.png
│       └── prediction_frequency.png
└── README.md
```

12. Next Steps (Beyond This Project)

- Streamlit deployment using MobileNetV2 (chosen over Xception for size/RAM efficiency on Streamlit Cloud's free tier; loaded directly from GitHub, no gdown needed)
- LinkedIn post summarizing the project, results, and key findings
- Move to NLP track: RNN/LSTM project for sequence/text data — continuing the "CNN → RNN/LSTM → further DS topics" learning roadmap

13. Key Takeaways / Lessons Learned

6. **Two independent HP tuning runs converging to the same accuracy ceiling is strong evidence the architecture itself is the bottleneck** — not worth further hyperparameter search at that point.
7. **Pretrained model input resolution matters a lot.** The EfficientNet family expects 224×224/300×300; feeding 150×150 caused severe underfitting (45–50% accuracy) regardless of fine-tuning strategy. Always check a pretrained model's expected input size before committing to it.
8. **shuffle=True on a tf.data.Dataset means every iteration re-shuffles** — never collect labels and predictions in separate passes over the same shuffled dataset.
9. **Val accuracy > Train accuracy isn't necessarily a red flag** — when data augmentation and regularization make training harder than the (unaugmented) validation set, this is expected and indicates good generalization, not a bug.
10. **High softmax confidence does not equal correctness.** Models can be 95–100% confident and wrong, especially on images with ambiguous/overlapping scene content. Confidence calibration differs meaningfully between architectures (MobileNetV2 vs. Xception in this project).
11. **Always clear notebook outputs before pushing API-key-containing cells to a public repository.**
12. **Smaller models can be “good enough.”** A 14MB model reaching 90.57% vs. a 112MB model reaching 91.73% — for most real deployment contexts, the smaller model is the better engineering choice.

Keep moving forward — from CNNs today, to RNNs, LSTMs, and beyond.